

# Dateien und Änderungen nachverfolgen

Git verstehen heißt nicht, Befehle auswendig zu lernen – es heißt, das **Modell dahinter** zu erkennen. Git speichert nicht einfach den letzten Stand, sondern hält **Snapshots** deines Projekts fest. Wer die Struktur versteht, dem werden fast alle Befehle logisch.

GIT GRUNDLAGEN



# Das Grundmodell: Drei Bereiche

Bevor du Befehle lernst, solltest du diese drei Bereiche kennen – sie sind das Herzstück von Git.

## Arbeitsverzeichnis

Die Dateien, an denen du gerade aktiv arbeitest.

## Staging Area (Index)

Ein Vorbereitungsbereich: Hier merkst du vor, was in den nächsten Commit soll.

## Repository

Die eigentliche Git-Historie mit allen gespeicherten Commits.

# Der Weg einer Änderung



Arbeitsverzeichnis

Staging Area

Repository

Die wichtigste Denkweise: **Bearbeiten** ist noch nicht **vormerken** – und vormerken ist noch nicht **dauerhaft sichern**. Erst `git commit` hält den Zustand wirklich in der Historie fest.

# Dateizustände in Git

Denke bei Git nicht zuerst in Befehlen, sondern in **Zuständen**. Frage dich immer: Liegt die Änderung im Arbeitsverzeichnis, in der Staging Area oder schon in einem Commit?

|                  |                                                   |
|------------------|---------------------------------------------------|
| <b>untracked</b> | Datei existiert, wird von Git noch nicht verfolgt |
| <b>modified</b>  | Datei wird verfolgt, wurde aber verändert         |
| <b>staged</b>    | Änderung ist für den nächsten Commit vorgemerkt   |
| <b>committed</b> | Änderung ist bereits in einem Commit gespeichert  |

# Dateien ins Repository aufnehmen

Neue Dateien sind zunächst **untracked** – Git kennt sie noch nicht. Der typische Ablauf für eine neue Datei `app.js`:

01

## Status prüfen

`git status` – Datei erscheint als *untracked*

02

## Vormerken

`git add app.js` – Datei wird in die Staging Area aufgenommen

03

## Committen

`git commit -m "Füge app.js hinzu"` – Zustand landet in der Repository-Historie

❏ Technisch präzise: `git add` nimmt die Datei in die **Staging Area** auf – erst `git commit` speichert sie in der **Repository-Historie**.

# Änderungen mit Commits sichern

## Was ist ein Commit?

Ein Commit ist ein festgehaltener Projektzustand mit einer Nachricht, die beschreibt, was geändert wurde. Er enthält **nur das, was vorher gestaged wurde** – nicht automatisch alle geänderten Dateien.

## Der große Vorteil

Du kannst bewusst entscheiden, welche Änderungen zusammengehören. Beispiel: Einen Rechtschreibfehler und eine neue Funktion **getrennt committen**, obwohl sie im selben Arbeitszeitraum entstanden sind. Das macht die Historie später viel verständlicher.

# Gute Commit-Nachrichten

Eine Commit-Nachricht sollte kurz und klar sagen, **was** geändert wurde. Je sauberer formuliert, desto leichter ist die Historie später nachvollziehbar.

## ✓ Gut

Behebe Fehler in der Passwortprüfung

Füge Formularvalidierung hinzu

Benenne Konfigurationsdatei um

## ✗ Weniger gut

Update

Änderungen

fix

# Arbeitsstatus und Historie prüfen

Bei Git reicht es nicht, nur Änderungen zu machen – du musst auch **lesen** können, was gerade los ist. Drei Befehle bilden dabei im Alltag eine starke Kombination:



## git status

Zeigt den Zustand des Arbeitsverzeichnisses und der Staging Area.



## git diff

Zeigt den genauen Inhalt der Änderungen – welche Zeilen sich verändert haben.



## git log

Zeigt die Commit-Historie mit Hash, Autor, Datum und Nachricht.



# git status richtig lesen

Ein typischer `git status`-Output zeigt gleichzeitig mehrere Zustände:

## **app.js – gestaged**

Unter „Changes to be committed“:  
bereits für den nächsten Commit  
vorgemerkt.

## **style.css – modified, nicht gestaged**

Unter „Changes not staged for  
commit“: verändert, aber noch nicht  
vorgemerkt.

## **notes.txt – untracked**

Unter „Untracked files“: neu, Git  
verfolgt sie noch nicht.

Mit `git status -s` erhältst du eine kompaktere Kurzform – im Alltag oft angenehmer.

# git diff und git log im Detail

| Frage                              | Befehl                         | Was er zeigt               |
|------------------------------------|--------------------------------|----------------------------|
| Welche Dateien sind betroffen?     | <code>git status</code>        | Zustand aller Dateien      |
| Welche Zeilen haben sich geändert? | <code>git diff</code>          | Nicht gestagete Änderungen |
| Was ist bereits gestaged?          | <code>git diff --staged</code> | Gestagete Änderungen       |
| Welche Commits gibt es?            | <code>git log</code>           | Vollständige Historie      |
| Kompakte Historienansicht          | <code>git log --oneline</code> | Hash + Nachricht pro Zeile |

📌 **Tipp:** `git log --oneline --graph --decorate --all` liefert eine besonders übersichtliche Darstellung der Historie mit Branch-Hinweisen.

## KAPITEL 3

# Änderungen rückgängig machen

Rückgängig machen ist in Git ein eigenes Thema. Die wichtigste Grundregel lautet: **Bevor du etwas rückgängig machst, musst du wissen, wo die Änderung gerade liegt.** Je nach Zustand braucht man ein anderes Werkzeug.



# git restore: Änderungen zurücksetzen

git restore stellt Inhalte im Arbeitsverzeichnis wieder her und kann mit Optionen auch die Staging Area beeinflussen.

## Arbeitsverzeichnis zurücksetzen

```
git restore app.js
```

Setzt die Datei auf den letzten bekannten Stand zurück. Nicht gespeicherte Änderungen gehen dabei **verloren**.

```
git restore .
```

 setzt alle Dateien zurück – Vorsicht!

## Staging Area zurücksetzen

```
git restore --staged app.js
```

Entfernt die Datei aus der Staging Area, ohne sie inhaltlich zu verändern. Extrem nützlich, wenn du aus Versehen etwas mit `git add` gestaged hast, das noch nicht in den nächsten Commit gehört.

## KAPITEL 4

# Branch-Management

Branches gehören zu den wichtigsten Git-Konzepten. Ein Branch ist im Kern ein **beweglicher Zeiger auf einen Commit**. Das macht Branches leichtgewichtig, schnell – und unverzichtbar für professionelle Entwicklung.

# Branches anzeigen und erstellen

## Branches anzeigen

`git branch` listet alle vorhandenen Branches auf. Das `*` markiert den aktuellen Branch:

`* main`

`feature-login`

`fix-header`

## Branch erstellen

`git branch feature-profilseite`

Erstellt einen neuen Branch – du bleibst aber **zunächst auf dem aktuellen Branch**. Das ist eine häufige Stolperfalle für Einsteiger.

Technisch wird kein Projektordner kopiert, sondern nur ein neuer Name erzeugt, der auf denselben Commit zeigt.

# Zwischen Branches wechseln

Zum Wechseln ist heute `git switch` die klarere Variante (ältere Anleitungen verwenden noch `git checkout`).

01

---

## Branch wechseln

`git switch feature-profilseite` – wechselt auf einen bestehenden Branch

02

---

## Erstellen und wechseln

`git switch -c feature-profilseite` – legt den Branch an und wechselt sofort dorthin

Beim Wechseln ändert Git deinen Arbeitskontext: Der Branch-Zeiger ändert sich, und Dateien im Arbeitsverzeichnis werden gegebenenfalls angepasst.

# Warum Branches unverzichtbar sind



## Features isoliert entwickeln

Neue Funktionen entstehen auf einem eigenen Branch – `main` bleibt sauber und stabil.



## Fehlerbehebungen isolieren

Bugfixes laufen auf separaten Branches, ohne laufende Feature-Entwicklung zu stören.



## Parallel arbeiten

Mehrere Themen gleichzeitig bearbeiten, ohne alles in einen chaotischen Verlauf zu mischen.

- ❑ Ein Branch ist **kein extra Projektordner**, sondern eine eigene Entwicklungslinie. Git wechselt leicht zwischen diesen Linien, weil Branches intern nur auf Commits zeigen.



# Ein realistisches Alltagsbeispiel

Typisches Setup mit drei parallelen Entwicklungslinien:

## **main**

Aktuell stabiler Stand – wird nicht  
direkt bearbeitet

## **feature-login**

Neue Login-Funktion – eigene  
Commits, unabhängig von main

## **fix-navbar**

Behebt einen Darstellungsfehler in der Navigation

# Alle wichtigen Befehle im Überblick

| Aufgabe                         | Befehl                                  |
|---------------------------------|-----------------------------------------|
| Status prüfen                   | <code>git status</code>                 |
| Datei vormerken                 | <code>git add datei</code>              |
| Commit erstellen                | <code>git commit -m "Nachricht"</code>  |
| Ungestagete Änderungen anzeigen | <code>git diff</code>                   |
| Gestagete Änderungen anzeigen   | <code>git diff --staged</code>          |
| Historie anzeigen               | <code>git log --oneline</code>          |
| Lokale Änderungen verwerfen     | <code>git restore datei</code>          |
| Datei aus Staging entfernen     | <code>git restore --staged datei</code> |
| Branches anzeigen               | <code>git branch</code>                 |
| Branch erstellen und wechseln   | <code>git switch -c name</code>         |

# Das Wichtigste: Struktur vor Befehlen

Nicht möglichst viele Befehle kennen, sondern die **Struktur des Systems** verstehen – das ist bei Git entscheidend.

## Drei Bereiche kennen

Arbeitsverzeichnis → Staging Area → Repository

## Zustände denken

Immer fragen: Wo liegt die Änderung gerade?

## Branches nutzen

Kein Luxus, sondern Kernwerkzeug professioneller Entwicklung